Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА №7

Бинарная классификация
Обобщённая задача XOR

Вариант 2

Студент ИУ8-13М (Группа) Бочкарев Михаил Александрович (И. О. Фамилия) _____ (Подпись, дата)

Преподаватель Басараб Михаил Алексеевич (И. О. Фамилия) _____ (Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы и постановка задачи 3
2 Модель сети и алгоритм обучения 4
2.1 Прямое распространение сигнала 4
2.2 Обратное распространение ошибки 4
2.3 Организация обучения и критерий остановки 5
3 Результаты численного эксперимента 6
3.1 Начальные параметры и первый шаг 6
3.2 Динамика обучения 7
3.3 Классификация образцов и разделяющая граница 7
4 Программная реализация 11
5 Выводы 12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 13
ПРИЛОЖЕНИЕ А Программа обучения и классификации 14
ПРИЛОЖЕНИЕ Б Программа построения графиков 16

1 Цель работы и постановка задачи

Цель работы — построить многослойную нейронную сеть прямого распространения и обучить её методом обратного распространения ошибки решению задачи бинарной классификации.

Согласно заданию [1] требуется использовать MLP архитектуры 2–2–1 и размеченные образцы варианта 2. Сеть содержит два входных признака, два скрытых нейрона и один выходной нейрон. Она называется двухслойной по числу слоёв обучаемых нейронов; входные узлы в этот подсчёт не включаются.

Каждый образец задаётся координатами (x_1, x_2) и меткой класса X или O. Горизонтальная координата x_1 возрастает слева направо. Вертикальная координата x_2 в таблице задания возрастает сверху вниз; эта ориентация сохранена на рисунке результатов. Пустые клетки не являются третьим классом и не включаются в обучающую выборку.

Разметка варианта 2 приведена в таблице 1. Используется 14 образцов: 6 класса X и 8 класса O. Нумерация образцов в программе соответствует чтению размеченных клеток по строкам сверху вниз и слева направо.

Таблица 1 – Разметка обучающих образцов варианта 2

$x_2 \backslash x_1$	0	0,25	0,5	0,75	1
0	X	–	–	O	O
0,25	O	X	X	–	O
0,5	–	O	X	X	–
0,75	–	–	–	X	O
1	O	–	–	O	–

Целевые значения кодируются как $t = +1$ для класса X и $t = -1$ для класса O. Предсказание класса выполняется по правилу

$$\widehat{c}(x_1, x_2) = \begin{cases} \text{X}, & y \geq 0, \\ \text{O}, & y < 0. \end{cases} \tag{1}$$

Кодировка классов, начальные веса, скорость обучения и численный порог ошибки выбраны для эксперимента: в задании [1] эти параметры не фиксированы.

2 Модель сети и алгоритм обучения

2.1 Прямое распространение сигнала

В соответствии с лекцией и примером Back Propagation [2, 3] используется нелинейная дифференцируемая функция активации

$$f(s) = \frac{1 - \exp(-s)}{1 + \exp(-s)} = \tanh\frac{s}{2}, \qquad f'(s) = \frac{1}{2}\bigl(1 - f^2(s)\bigr). \tag{2}$$

Линейные функции активации всех слоёв свели бы сеть к линейному отображению. Нелинейность необходима для построения нелинейной границы классов. В отличие от нейрона высокого порядка, в данной модели не добавляются произведения входов: архитектура остаётся 2–2–1.

Обозначим вес входа i скрытого нейрона j через a_{ij}, а веса выходного нейрона через b_0, b_1, b_2. Постоянный вход смещения равен $x_0 = 1$. Прямой проход имеет вид

$$net_j = a_{0j} + a_{1j}x_1 + a_{2j}x_2, \qquad h_j = f(net_j), \quad j = 1, 2, \tag{3}$$

$$net_{out} = b_0 + b_1 h_1 + b_2 h_2, \qquad y = f(net_{out}). \tag{4}$$

У каждого скрытого нейрона три параметра, включая смещение; у выходного нейрона также три параметра. Общее число обучаемых параметров равно $2 \cdot 3 + 3 = 9$.

2.2 Обратное распространение ошибки

Для одного образца минимизируется функция потерь $L = \frac{1}{2}(t - y)^2$. Сначала вычисляются сигналы ошибки выходного и обоих скрытых нейронов:

$$\delta_{out} = \frac{1}{2}(1 - y^2)(t - y), \tag{5}$$

$$\delta_j = \frac{1}{2}(1 - h_j^2) b_j \delta_{out}, \qquad j = 1, 2. \tag{6}$$

Все величины в этих формулах определяются по весам одного и того же прямого прохода. Обновление весов начинается только после вычисления всех трёх сигналов ошибки:

$$a_{ij} \leftarrow a_{ij} + \eta x_i \delta_j, \qquad i = 0, 1, 2, \quad j = 1, 2, \tag{7}$$

$$b_0 \leftarrow b_0 + \eta \delta_{out}, \qquad b_j \leftarrow b_j + \eta h_j \delta_{out}. \tag{8}$$

Знак «плюс» соответствует определению ошибки как $t - y$. При вычислении скрытых ошибок используются старые выходные веса, а для обновления выходных весов — скрытые сигналы, сохранённые во время прямого прохода.

2.3 Организация обучения и критерий остановки

Одна эпоха включает однократное предъявление всех 14 образцов. Веса обновляются после каждого образца; порядок предъявления перемешивается перед каждой эпохой. После полного прохода выходы для всех образцов рассчитываются заново при фиксированных текущих весах. По ним определяются

$$E(k) = \sqrt{\sum_{p=1}^{14} \big(t_p - y_p(k)\big)^2}, \qquad \mathrm{MSE}(k) = \frac{E^2(k)}{14}, \tag{9}$$

а также число неверных классификаций $N_{err}(k)$.

Параметры эксперимента: $\eta = 0{,}1$, $\varepsilon = 0{,}1$, предельное число эпох — 100000. Обучение завершается при одновременном выполнении условий

$$N_{err}(k) = 0, \qquad E(k) \le \varepsilon. \tag{10}$$

Состояние $k = 0$ фиксируется до первого обновления. Начальные веса выбираются генератором `Random(2)` равномерно из $[-1, 1]$; тот же экземпляр генератора используется для перемешивания. Это обеспечивает воспроизводимость запуска. Ненулевые различные веса скрытых нейронов предотвращают сохранение симметрии при обучении.

При выбранной активации точные значения ± 1 достигаются только в пределе, поэтому проверяется конечная допустимая ошибка, а не точное равенство $y = t$. При пообразцовом обучении ошибка всей выборки не обязана уменьшаться после каждого обновления или каждой эпохи.

3 Результаты численного эксперимента

3.1 Начальные параметры и первый шаг

Начальные и конечные веса приведены в таблице 2. В таблицах значения округлены; расчёт и проверка критерия остановки выполняются без предварительного округления.

Таблица 2 – Начальные и итоговые веса сети

Параметр	$k = 0$	$k = 7816$
a_{01}	0.912069	-7.204302
a_{11}	0.895655	11.658693
a_{21}	-0.886897	-4.401201
a_{02}	-0.830256	-0.753576
a_{12}	0.670998	-9.107400
a_{22}	0.471940	8.521447
b_0	0.339461	-8.015964
b_1	-0.383727	-9.242022
b_2	0.211888	-9.687032

После первого перемешивания первым предъявляется образец № 13: $(x_1, x_2) = (0, 1)$, класс O, $t = -1$. Прямой проход даёт

$$(h_1, h_2) \approx (0.012585;\ -0.177265), \qquad y \approx 0.147453.$$

Сигналы ошибки равны

$$\delta_{out} \approx -0.561252, \qquad (\delta_1, \delta_2) \approx (0.107667;\ -0.057593).$$

Для проверки первого обновления параметров получаем

$$(a_{01}, a_{11}, a_{21}) \approx (0.922835;\ 0.895655;\ -0.876131),$$

$$(a_{02}, a_{12}, a_{22}) \approx (-0.836015;\ 0.670998;\ 0.466181),$$

$$(b_0, b_1, b_2) \approx (0.283336;\ -0.384433;\ 0.221837).$$

Так как у этого образца $x_1 = 0$, веса a_{11} и a_{12} на первом шаге не изменяются; остальные поправки вычисляются по приведённым формулам. Файл `first_epoch.csv` содержит сигналы и веса до и после каждого из 14 обновлений первой эпохи.

3.2 Динамика обучения

В таблице 3 показаны выбранные эпохи. Полные результаты для всех эпох, включая девять весов и 14 выходов сети, сохранены в `history.csv`. На рисунке 1 приведены ошибка и число неверных классификаций.

Таблица 3 – Изменение ошибки и числа неверных классификаций

k	$E(k)$	MSE	N_{err}
0	3.80529731	1.03430626	8
1	3.76792758	1.01409130	7
10	3.71078135	0.98356416	6
100	3.62498418	0.93860788	5
500	3.62207810	0.93710356	6
1000	3.61588009	0.93389920	5
2000	3.60362140	0.92757766	5
2609	2.34581010	0.39305893	1
2610	2.32076623	0.38471113	0
3000	0.52414217	0.01962322	0
4000	0.22383313	0.00357866	0
5000	0.15953145	0.00181788	0
6000	0.12902972	0.00118919	0
7000	0.11058329	0.00087348	0
7815	0.10000721	0.00071439	0
7816	0.09999591	0.00071423	0

Впервые все 14 точек классифицированы верно на эпохе 2610, однако величина ошибки ещё составляет примерно 2,320766. Поэтому обучение продолжается. Условие $E \le 0{,}1$ впервые достигнуто на эпохе 7816:

$$E(7815) \approx 0{,}10000721 > 0{,}1,$$

$$E(7816) \approx 0{,}09999591 < 0{,}1, \qquad N_{err}(7816) = 0.$$

За 7816 эпох выполнено $7816 \cdot 14 = 109424$ пообразцовых обновления. Количество строк истории с учётом исходного состояния равно 7817.

3.3 Классификация образцов и разделяющая граница

Итоговые ответы приведены в таблице 4. Все заданные метки воспроизведены верно. Значение выхода используется как числовая оценка класса и не интерпретируется как вероятность.

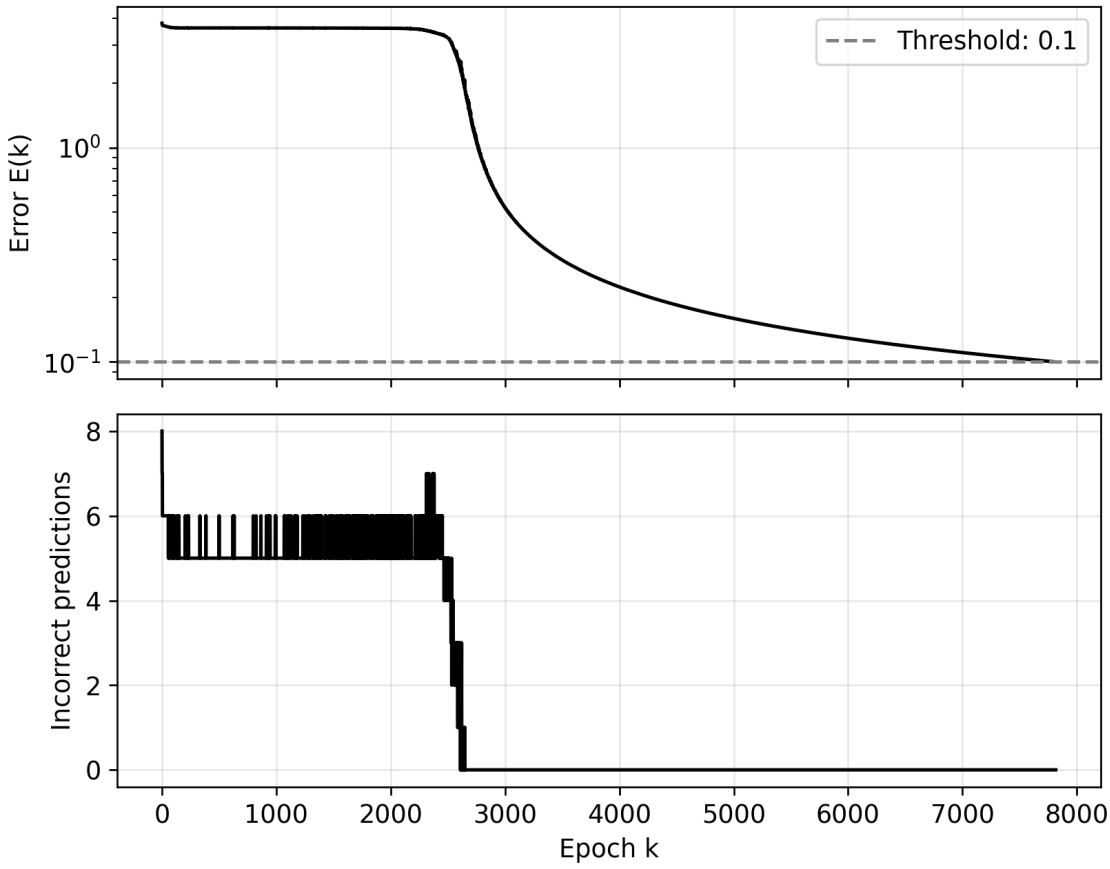


Рисунок 1 – Ошибка и число неверных классификаций по эпохам

Таблица 4 – Ответы обученной сети на всех размеченных образцах

№	x_1	x_2	t	y	Цель	Ответ
1	0.00	0.00	1	0.981953	X	X
2	0.75	0.00	-1	-0.973670	O	O
3	1.00	0.00	-1	-0.998728	O	O
4	0.00	0.25	-1	-0.979317	O	O
5	0.25	0.25	1	0.989343	X	X
6	0.50	0.25	1	0.999668	X	X
7	1.00	0.25	-1	-0.998095	O	O
8	0.25	0.50	-1	-0.967649	O	O
9	0.50	0.50	1	0.990777	X	X
10	0.75	0.50	1	0.962022	X	X
11	0.75	0.75	1	0.951458	X	X
12	1.00	0.75	-1	-0.952783	O	O
13	0.00	1.00	-1	-0.999574	O	O
14	0.75	1.00	-1	-0.964313	O	O

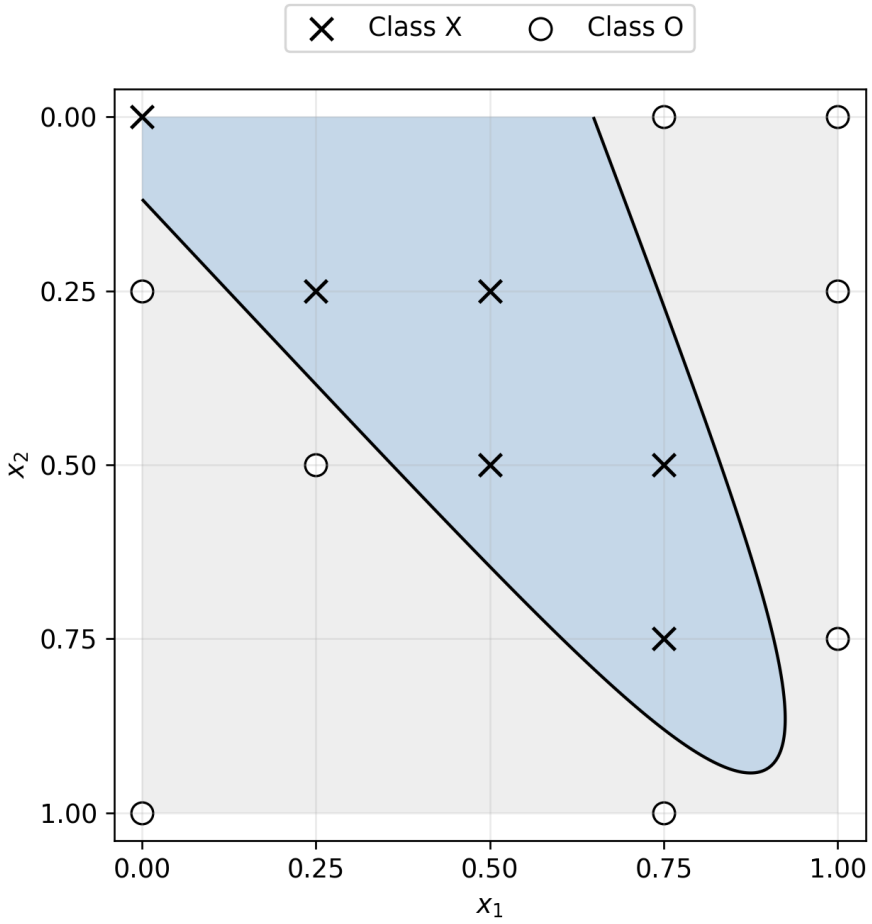


Рисунок 2 – Разделение пространства на классы: граница соответствует $y = 0$

На рисунке 2 светло-серая область соответствует классу O, голубая — классу X. Крестики и кружки обозначают исходные размеченные образцы. Направление вертикальной оси совпадает с таблицей задания: x_2 увеличивается вниз.

Граница $y = 0$ получена из выхода обученной сети на равномерной сетке. Цвет в пустых клетках показывает предсказание модели, но не подтверждённую истинную метку. Отдельная тестовая выборка в задании не предоставлена, поэтому точность $100\,\%$ относится только к обучающим образцам.

4 Программная реализация

Обучение реализовано на Python 3 без библиотек машинного обучения. Программа `train.py` использует стандартные модули `math`, `random`, `csv`, `json` и `pathlib`. Полный код приведён в приложении А, листинг А.1.

Функция `forward` выполняет прямой проход; `deltas` вычисляет ошибки обоих слоёв по старым весам; `evaluate` оценивает всю выборку без изменения параметров. В основном цикле после вычисления ошибок обновляются веса скрытого и выходного слоёв. Генератор псевдослучайных чисел используется с фиксированным seed.

Для запуска обучения используется команда `python train.py`. Результаты сохраняются рядом со скриптом: `samples.csv` содержит обучающую выборку; `history.csv` — историю всех эпох; `first_epoch.csv` — подробности первой эпохи; `predictions.csv` — итоговые ответы; `model.json` — начальные и конечные параметры и настройки обучения.

Вспомогательный скрипт `plot_results.py` строит графики по сохранённым результатам и приведён в приложении Б, листинг Б.2. Для него требуются NumPy и Matplotlib. Изображения уже включены в проект; для сборки отчёта запуск Python не нужен.

5 Выводы

Построена и обучена двухслойная MLP архитектуры 2–2–1 для варианта 2 обобщённой задачи XOR. Реализованы нелинейная активация, обратное распространение ошибок и обновление всех девяти параметров сети.

При скорости обучения $\eta = 0{,}1$ и фиксированном seed 2 первая безошибочная классификация всей обучающей выборки получена на эпохе 2610. Численный критерий $E \leq 0{,}1$ достигнут на эпохе 7816; итоговая ошибка $E \approx 0{,}09999591$. Правильно классифицированы все 14 образцов.

Эксперимент демонстрирует различие между верным определением класса по знаку выхода и приближением выхода к числовой цели. Построенная граница согласуется с размеченными образцами. Обобщающая способность сети на новых размеченных данных в этой работе не оценивалась.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

а) Решение задачи бинарной классификации на примере обобщённой задачи XOR. Задание и варианты работы, файл `BP_pattern_variants.pdf`. Вариант 2.

б) Многослойные НС прямого распространения. Учебная презентация, файл `L4a_Многослойные.pdf`. 29 слайдов.

в) Басараб М. А., Коннова Н. С. Метод обратного распространения ошибки (Back Propagation). Пример: архитектура 1–1–2 (одна эпоха обучения). МГТУ им. Н. Э. Баумана, кафедра ИУ8. 11 слайдов.

ПРИЛОЖЕНИЕ А

Программа обучения и классификации

Листинг А.1 – Обучение сети MLP 2–2–1

```
from math import tanh, sqrt
from pathlib import Path
from random import Random
import csv
import json

ROOT = Path(__file__).resolve().parent
DATA = [(0.00, 0.00, 1), (0.75, 0.00, -1), (1.00, 0.00, -1)]
DATA += [(0.00, 0.25, -1), (0.25, 0.25, 1), (0.50, 0.25, 1)]
DATA += [(1.00, 0.25, -1), (0.25, 0.50, -1), (0.50, 0.50, 1)]
DATA += [(0.75, 0.50, 1), (0.75, 0.75, 1), (1.00, 0.75, -1)]
DATA += [(0.00, 1.00, -1), (0.75, 1.00, -1)]
ETA = 0.1
EPSILON = 0.1
SEED = 2
MAX_EPOCHS = 100000

def activation(net):
    return tanh(net / 2.0)

def derivative(value):
    return 0.5 * (1.0 - value * value)

def forward(point, hidden_weights, output_weights):
    inputs = [1.0, point[0], point[1]]
    nets = [sum(a * x for a, x in zip(w, inputs)) for w in hidden_weights]
    hidden = [activation(net) for net in nets]
    net = output_weights[0] + sum(w * h for w, h in zip(output_weights[1:], hidden))
    return inputs, hidden, activation(net)

def deltas(target, hidden, output, output_weights):
    output_delta = derivative(output) * (target - output)
    hidden_delta = [derivative(h) * w * output_delta for h, w in zip(hidden, output_weights[1:])]
    return hidden_delta, output_delta

def evaluate(hidden_weights, output_weights):
    outputs = [forward(p, hidden_weights, output_weights)[2] for p in DATA]
    error = sqrt(sum((p[2] - y) ** 2 for p, y in zip(DATA, outputs)))
    wrong = sum((1 if y >= 0 else -1) != p[2] for p, y in zip(DATA, outputs))
    return outputs, error, wrong

def flatten(hidden_weights, output_weights):
    return [a for row in hidden_weights for a in row] + list(output_weights)

def save_csv(name, header, rows):
    with (ROOT / name).open("w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(header)
        writer.writerows(rows)

def main():
    random = Random(SEED)
    hidden_weights = [[random.uniform(-1.0, 1.0) for _ in range(3)] for _ in range(2)]
    output_weights = [random.uniform(-1.0, 1.0) for _ in range(3)]
    initial_weights = flatten(hidden_weights, output_weights)
    history = []
    trace = []
    first_correct = None
    order = list(range(len(DATA)))
    for epoch in range(MAX_EPOCHS + 1):
        outputs, error, wrong = evaluate(hidden_weights, output_weights)
        weights = flatten(hidden_weights, output_weights)
        history.append([epoch, error, error ** 2 / len(DATA), wrong, *weights, *outputs])
        if wrong == 0 and first_correct is None:
            first_correct = epoch
        if epoch % 1000 == 0 or (wrong == 0 and error <= EPSILON):
            print(f"epoch={epoch}; E={error:.8f}; wrong={wrong}/{len(DATA)}")
        if wrong == 0 and error <= EPSILON:
            break
        if epoch == MAX_EPOCHS:
            raise RuntimeError("Training did not reach the stopping criterion")
        random.shuffle(order)
        for step, index in enumerate(order, 1):
            point = DATA[index]
            inputs, hidden, output = forward(point, hidden_weights, output_weights)
```

```
            hidden_delta, output_delta = deltas(point[2], hidden, output, output_weights)
            before = flatten(hidden_weights, output_weights)
            for j in range(2):
                for i in range(3):
                    hidden_weights[j][i] += ETA * inputs[i] * hidden_delta[j]
            output_weights[0] += ETA * output_delta
            for j in range(2):
                output_weights[j + 1] += ETA * hidden[j] * output_delta
            if epoch == 0:
                after = flatten(hidden_weights, output_weights)
                signals = [*point, *hidden, output, *hidden_delta, output_delta]
                trace.append([step, index + 1, *signals, *before, *after])
    names = ["a01", "a11", "a21", "a02", "a12", "a22", "b0", "b1", "b2"]
    header = ["epoch", "E", "MSE", "wrong"] + names
    header += [f"y{i + 1}" for i in range(len(DATA))]
    save_csv("history.csv", header, history)
    header = ["step", "sample", "x1", "x2", "t", "h1", "h2", "y", "d1", "d2", "dout"]
    header += ["before_" + name for name in names] + ["after_" + name for name in names]
    save_csv("first_epoch.csv", header, trace)
    predictions = []
    for i, (point, output) in enumerate(zip(DATA, outputs), 1):
        target = "X" if point[2] == 1 else "O"
        predicted = "X" if output >= 0 else "O"
        predictions.append([i, *point, output, target, predicted, target == predicted])
        print(f"{i:2d}: x={point[:2]}; target={target}; y={output:.8f}; class={predicted}")
    header = ["id", "x1", "x2", "t", "y", "target_class", "predicted_class", "correct"]
    save_csv("predictions.csv", header, predictions)
    save_csv("samples.csv", ["id", "x1", "x2", "t"], [[i, *p] for i, p in enumerate(DATA, 1)])
    summary = {"seed": SEED, "eta": ETA, "epsilon": EPSILON, "epochs": epoch}
    summary.update({"E": error, "wrong": wrong, "first_correct_epoch": first_correct})
    summary.update({"initial_weights": initial_weights, "hidden_weights": hidden_weights})
    summary.update({"output_weights": output_weights, "weight_names": names})
    (ROOT / "model.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print(f"First epoch with no classification errors: {first_correct}")
    print(f"Hidden weights: {hidden_weights}")
    print(f"Output weights: {output_weights}")

if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ Б

Программа построения графиков

Листинг Б.2 – Визуализация результатов обучения

```
from pathlib import Path
import csv
import json
import matplotlib.pyplot as plt
import numpy as np

ROOT = Path(__file__).resolve().parent
with (ROOT / "history.csv").open(encoding="utf-8") as file:
    history = list(csv.DictReader(file))
with (ROOT / "predictions.csv").open(encoding="utf-8") as file:
    samples = list(csv.DictReader(file))
model = json.loads((ROOT / "model.json").read_text(encoding="utf-8"))
epochs = [int(row["epoch"]) for row in history]
errors = [float(row["E"]) for row in history]
wrong = [int(row["wrong"]) for row in history]
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11})
fig, axes = plt.subplots(2, 1, figsize=(7.0, 5.5), sharex=True)
axes[0].semilogy(epochs, errors, color="black")
axes[0].axhline(model["epsilon"], color="gray", linestyle="--", label="Threshold: 0.1")
axes[0].set_ylabel("Error E(k)")
axes[0].legend()
axes[1].plot(epochs, wrong, color="black")
axes[1].set_xlabel("Epoch k")
axes[1].set_ylabel("Incorrect predictions")
for ax in axes:
    ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig(ROOT / "learning.png", dpi=240)
plt.close(fig)
coordinates = np.linspace(0.0, 1.0, 401)
x1, x2 = np.meshgrid(coordinates, coordinates)
hidden = []
for bias, w1, w2 in model["hidden_weights"]:
    hidden.append(np.tanh((bias + w1 * x1 + w2 * x2) / 2.0))
b0, b1, b2 = model["output_weights"]
output = np.tanh((b0 + b1 * hidden[0] + b2 * hidden[1]) / 2.0)
fig, ax = plt.subplots(figsize=(6.2, 5.7))
ax.contourf(x1, x2, output, levels=[-1, 0, 1], colors=["#eeeeee", "#c5d7e8"])
ax.contour(x1, x2, output, levels=[0], colors="black", linewidths=1.3)
for label, marker in [("X", "x"), ("O", "o")]:
    points = [row for row in samples if row["target_class"] == label]
    xs = [float(row["x1"]) for row in points]
    ys = [float(row["x2"]) for row in points]
    if label == "X":
        ax.scatter(xs, ys, marker=marker, s=90, color="black", label="Class X", zorder=3)
    else:
        ax.scatter(xs, ys, s=85, facecolors="white", edgecolors="black", label="Class O")
ax.set_xlim(-0.04, 1.04)
ax.set_ylim(1.04, -0.04)
ax.set_xticks([0, 0.25, 0.5, 0.75, 1])
ax.set_yticks([0, 0.25, 0.5, 0.75, 1])
ax.set_xlabel(r"$x_1$")
ax.set_ylabel(r"$x_2$")
ax.set_aspect("equal")
ax.grid(True, alpha=0.25)
ax.legend(loc="upper center", bbox_to_anchor=(0.5, 1.13), ncol=2)
fig.tight_layout()
fig.savefig(ROOT / "decision.png", dpi=240)
plt.close(fig)
```